

Rapport de TP1 : Analyse et traitement des images

Instrumentation 2 : 2024 /2025

SINACER Tedj El Moulk

DAHMOUN Sarah

Ce TP vise à nous initier aux techniques d'analyse et de traitements d'images à l'aide de l'environnement python, en utilisant l'éditeur Spyder. Nous avons travaillé avec plusieurs bibliothèques, comme Numpy, matplotlib, ainsi que des outils spécifiques à l'image tel que Skimage et OpenCv. Ce TP traite plusieurs aspects fondamentaux du traitement de l'image, tel que la lecture d'images dans différents formats, la manipulation des pixels, l'analyse des espaces de couleurs (RGB, LAB, HSV), et les opérations sur les images en niveaux de gris et de couleur.

Nous avons également réalisé une analyse fréquentielle de l'image en utilisant la transformée de Fourier, ainsi que des techniques de filtrage spatiale pour la détection de contours. Ce TP nous a donc permis d'acquérir une compréhension approfondie des concepts clés en traitement d'images.

A- Lecture des images sous Python

Les images sont sauvegardées dans des fichiers graphiques. Il existe plusieurs formats de fichier : bmp, tif, jpeg, Nous avons eu à disposition plusieurs images et nous avons utiliser l'image flowers.tif .

Exercice 1 : Dimension des images

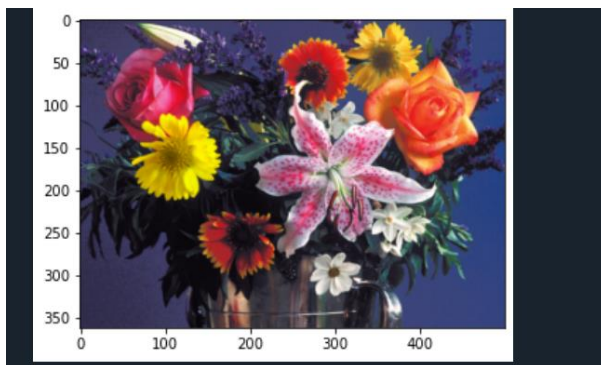
Dans cette partie nous avons utiliser la fonction `io.imread` pour pouvoir lire des images. Pour y faire nous avons fait appel à la bibliothèque Skimage (Pour lire l'image) et matplotlib (utilisé pour la visualisation sous formes de graphiques et figures).

1- Pour lire le fichier nous avons utilisé cette partie de code

```
from skimage import io
from matplotlib import pyplot as plt

image = io.imread('flowers.tif',False)
plt.imshow(image)
```

La fonction `io.imread` permet de lire l'image, tandis que la fonction `plt.imshow` nous a permis d'afficher ceci :



Pour se débarrasser des axes nous avons utilisé cette fonction :

```
plt.axis('off')
```

Nous avons donc finalement obtenu ceci :



Nous avons dans la fonction modifié le False par True comme suit :

```
from skimage import io
from matplotlib import pyplot as plt

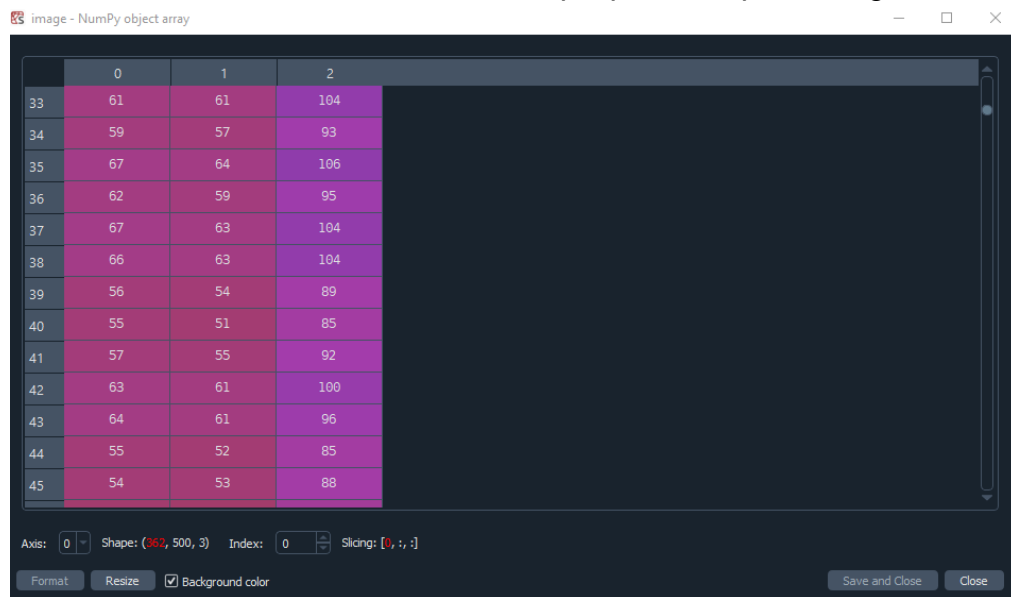
image = io.imread('flowers.tif', True)
plt.imshow(image)
plt.axis('off')
```

Ce code nous a permis d'afficher ceci

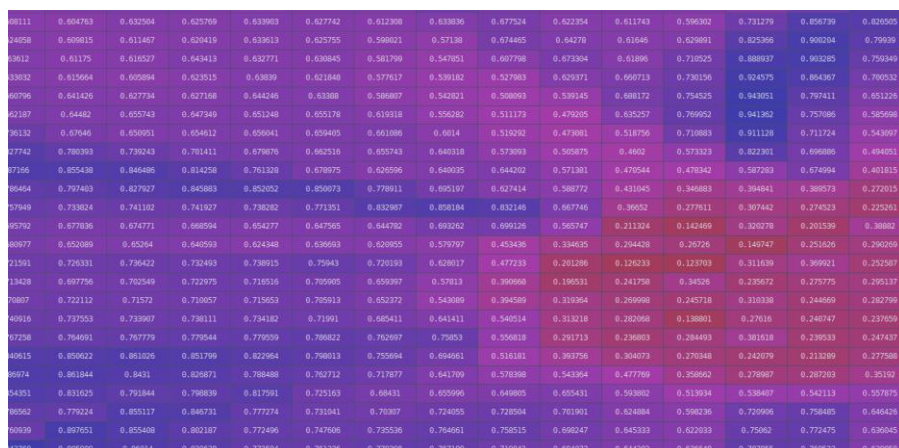


a) On en déduit donc que lorsqu'on utilise la fonction `io.imread (image, argument)`, Si l'argument est `True`, la lecture de l'image est en couleur (mode RGB), si l'argument est `False` l'image est lue en niveaux .

Quand on lit la matrice dans le workspace, la matrice de l'image en couleur me donne taille de la matrice et le 3, qui précise que l'image est en mode RGB :



Sinon elle donne juste la taille de la matrice et c'est la même taille dans les deux cas :



b) La déduction des données, afin d'afficher cette information nous utilisons ce code :

Nous avons donc fait appel à une autre bibliothèque numpy qui permet d'utiliser les fonctions image.dtype et image.shape

```
from skimage import io
from matplotlib import pyplot as plt
import numpy as np

image = io.imread('flowers.tif', False)
print(" le type de données :", image.dtype)
print(image.shape)
```

Le résultat est donc le type de données : (unit8) unsigned integer 8bits (false RGB), ainsi que la taille de l'image (362, 500). Ce type représente un entier non signé sur 8bits, cet entier est compris entre 0 et 255. C'est-à-dire chaque pixel a une valeurs qui est comprise entre 0 et 255, ce qui correspond à une couleur dans un canal RGB .

```
from skimage import io
from matplotlib import pyplot as plt
import numpy as np

image = io.imread('flowers.tif',True)
print(" le type de données :", image.dtype)
print(image.shape)
```

```
In [52]: runfile('C:/Users/12108993/Desktop/tp-ti/tp1/untitled1.py', wdir='C:/Users/12108993/Desktop/tp-ti/tp1')
le type de données : float64
(362, 500)
```

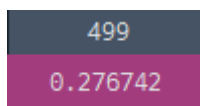
Dans le cas d'une image lue en niveaux de gris le type de données est float64 pour donc entier avec virgule 64 bits sont utilisés pour stocker un nombre. Cette image contient des valeurs entre 0.0 et 1.0, c'est-à-dire les valeurs ont été normalisé et comme nous sommes dans le cas de l'image modifiée , les valeurs sont très cohérentes.

Malgré la modification de l'image, et le changement dans le type des données, la taille de l'image ne change pas .

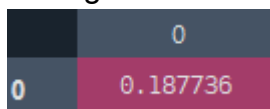
Ce type de données est très cohérent avec ce que nous avons pu observer dans le workspace, c'est-à-dire avec les valeurs des données des matrices que nous avons affichées auparavant.

- 2- Pour afficher les coins de l'image nous avons utilisé le workspace, on sait que la matrice est de taille (362,500) , donc les valeurs de l'image vont d'un indice 0 à 361 et 0 à 500 dans les matrices.

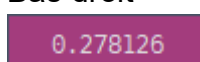
Haut droit



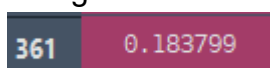
Haut gauche



Bas droit



Bas gauche



3- Si on utilise la fonction cv.imread il faut d'abord appeler la bibliothèque CV2

```
import cv2
from skimage import io
from matplotlib import pyplot as plt
import numpy as np

image = cv2.imread('flowers.tif')
```

La principale différence entre cv2.imread et io.imread est que cv2.imread charge les images en BGR par défaut, tandis que io.imread les charge en RGB. Quand nous avons afficher la matrice de l'image chargé avec io.imread puis avec cv2.imread nous avons remarqué que les valeurs des canaux de couleur sont inversées pour les canaux bleu et rouge .

Exercice 2 : Espace des couleurs

1- Extraction des trois composantes de l'image dans l'espace de couleurs RGB :

```
image = cv2.imread('flowers.tif')
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

R, G, B = cv2.split(image_rgb)
R_image = cv2.merge([R, R*0, R*0])
G_image = cv2.merge([G*0, G, G*0])
B_image = cv2.merge([B*0, B*0, B])

plt.figure(figsize=(15, 5))

plt.subplot(1,3,1)
plt.imshow(R_image)
plt.title('canal rouge')
plt.axis('off')

plt.subplot(1,3,2)
plt.imshow(G_image)
plt.title('canal vert')
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(B_image)
plt.title('canal bleu')
plt.axis('off')

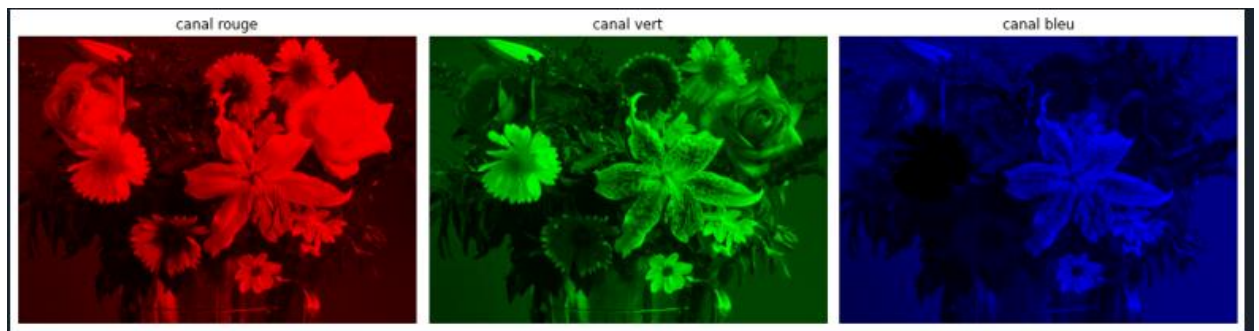
plt.tight_layout()
plt.show()
```

Par défaut openCV charge les images en BGR donc nous avons converti notre image en RGB avec la fonction « image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB) ».

Nous avons ensuite séparé les canaux Rouge, Vert, Bleu, c'est-à-dire nous avons divisé l'image en trois matrices distinctes correspondant aux trois canaux de couleur : rouge, vert, bleu. Avec la fonction « `cv2.split()` »

Nous avons après ceci créé une image pour chaque canal avec la fonction « `cv2.merge()` ». Pour le canal rouge, nous avons gardé uniquement la composante rouge, tandis que les composantes verte et bleue sont mises à zéro. De même, les canaux vert et bleu.

Finalement nous avons affiché les canaux avec Matplotlib, qui crée un sous graphe pour afficher chaque canal comme suit :



Ce que nous avons observé :

- **Canal Rouge** : cette image montre l'intensité de la couleur rouge. Les zones où le rouge est dominant apparaîtront plus brillantes.
- **Canal Vert** : cette image montre l'intensité de la couleur verte, avec les zones dominées par le vert qui apparaîtront brillantes.
- **Canal Bleu** : cette image montre l'intensité de la couleur bleue, avec les zones où le bleu est dominant qui apparaîtront brillantes.

2- Extraction des trois composantes de l'image dans l'espace de couleurs LAB :

L'espace de couleurs LAB est un modèle colorimétrique perceptuel qui décompose la couleur en trois composantes : L pour la luminosité, et A et B pour les dimensions chromatiques (rouge-vert pour A et bleu-jaune pour B).

Pour pouvoir extraire ces composantes nous avons utilisé le code suivant :

```

import cv2
from skimage import io
from matplotlib import pyplot as plt
import numpy as np

image = cv2.imread('flowers.tif')
image_lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

L, A, B = cv2.split(image_lab)

plt.figure(figsize =(15, 5))

plt.subplot(1,3,1)
plt.imshow(L)
plt.title('canal L(luminosite)')
plt.axis('off')

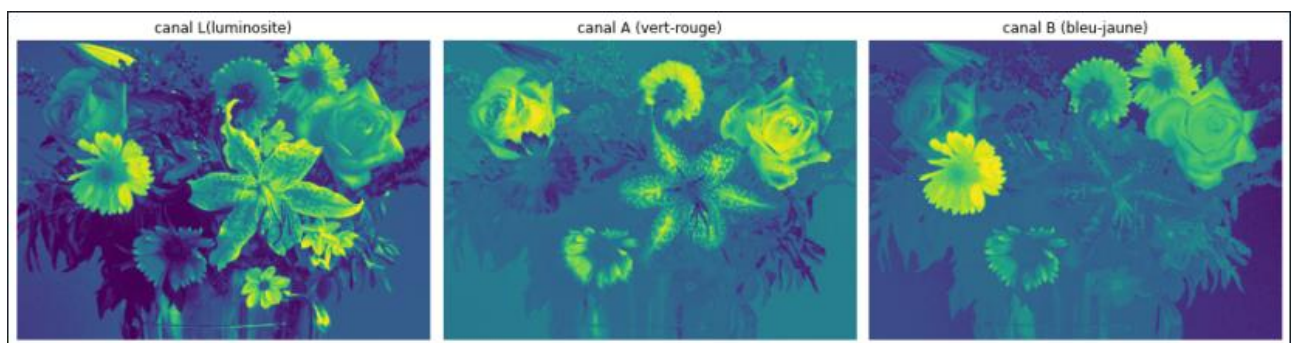
plt.subplot(1,3,2)
plt.imshow(A)
plt.title('canal A (vert-rouge)')
plt.axis('off')

plt.subplot(1,3,3)
plt.imshow(B)
plt.title('canal B (bleu-jaune)')
plt.axis('off')

plt.tight_layout()
plt.show()

```

Ce code Python utilise OpenCV pour charger une image, celle-ci est chargée dans l'espace de couleur BGR, nous l'avons donc converti à l'espace de couleur LAB. Ensuite nous avons extrait les canaux L,A et B. Finalement nous avons affiché chaque canal séparément dans une figure, comme suit :



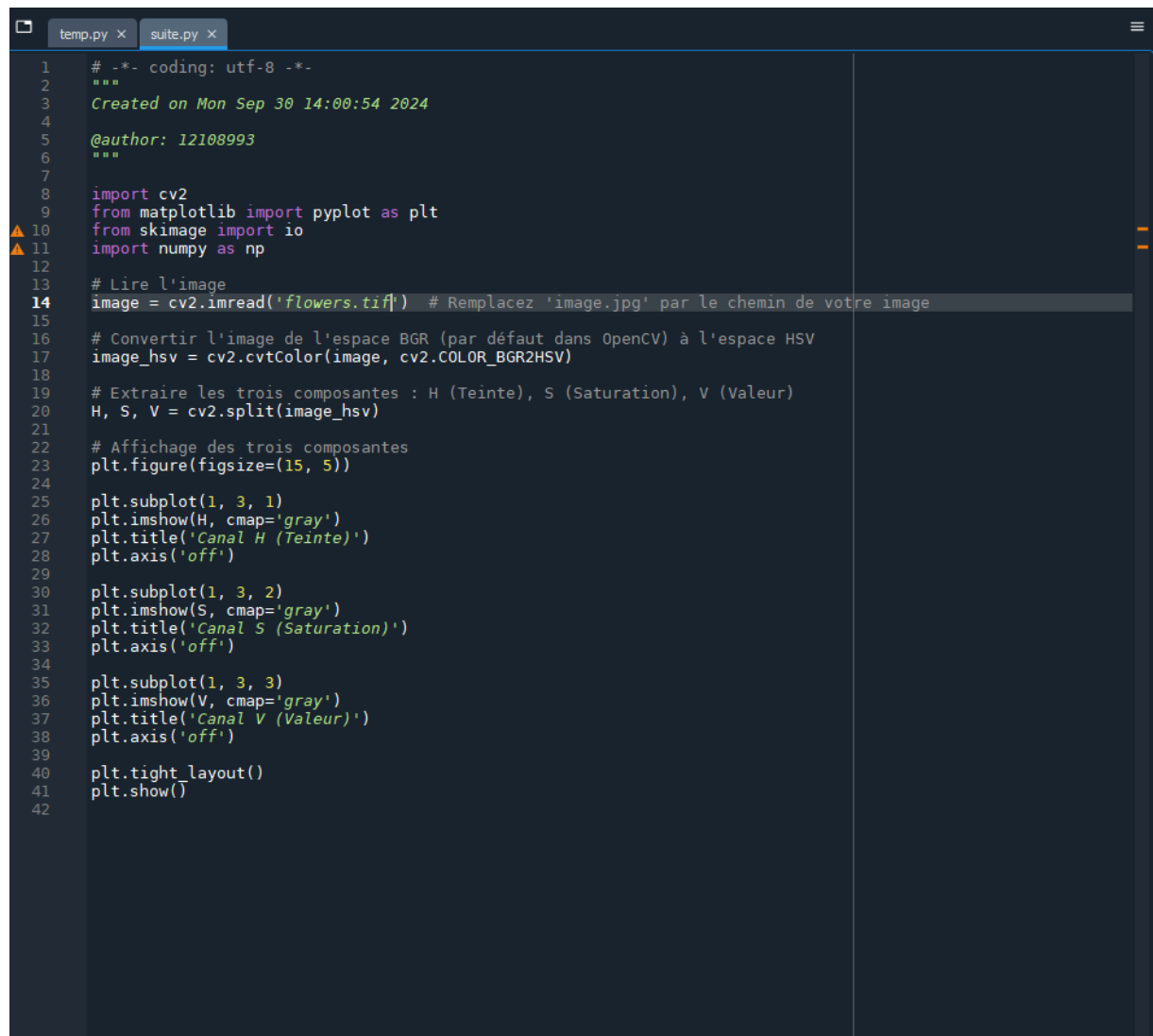
Ce que nous avons observé :

Canal L : les zones plus claires représentent des parties plus lumineuses, tandis que les zones plus sombres sont des parties moins éclairées.

Canal A (Vert-Rouge) : les zones vertes apparaissent avec une intensité différente par rapport aux zones rouges. Les zones plus claires tendent vers le rouge, et les zones plus sombres tendent vers le vert.

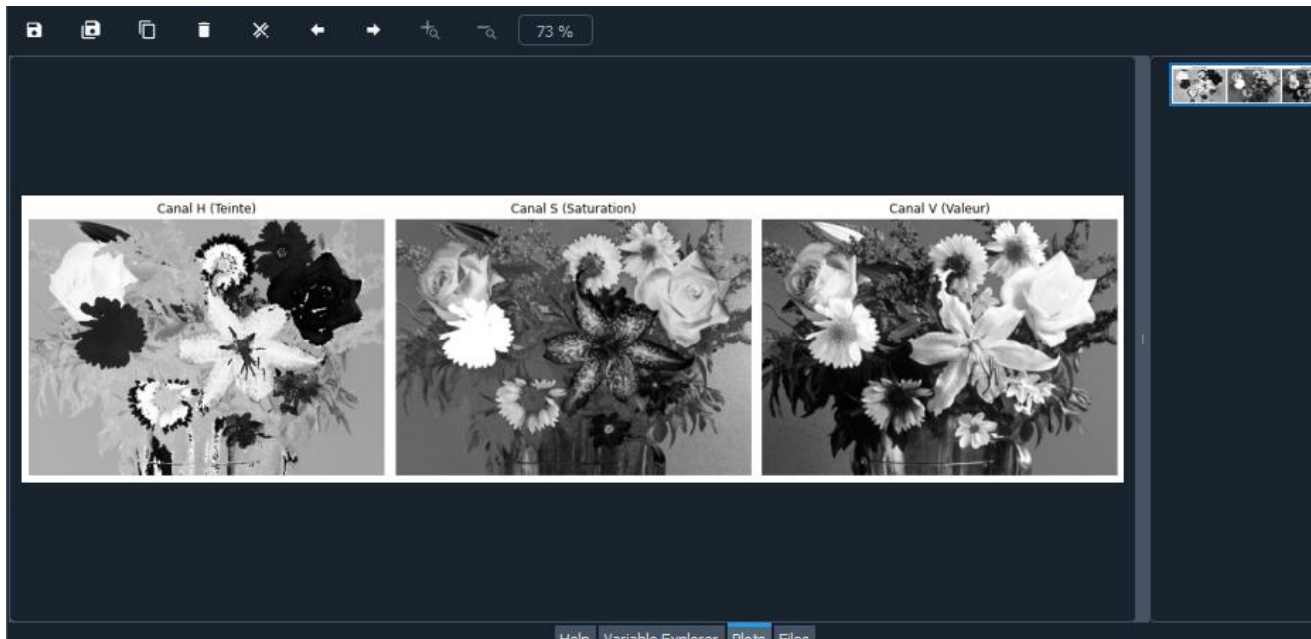
Canal B (Bleu- Jaune) : les zones claires représentent des parties tirant vers le jaune, et les zones sombres indiquent des parties plus bleues.

3- Extraction des trois composantes de l'image dans l'espace de couleurs LAB :



```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Mon Sep 30 14:00:54 2024
4
5  @author: 12108993
6  """
7
8  import cv2
9  from matplotlib import pyplot as plt
10 from skimage import io
11 import numpy as np
12
13 # Lire l'image
14 image = cv2.imread('flowers.tif') # Remplacez 'image.jpg' par le chemin de votre image
15
16 # Convertir l'image de l'espace BGR (par défaut dans OpenCV) à l'espace HSV
17 image_hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
18
19 # Extraire les trois composantes : H (Teinte), S (Saturation), V (Valeur)
20 H, S, V = cv2.split(image_hsv)
21
22 # Affichage des trois composantes
23 plt.figure(figsize=(15, 5))
24
25 plt.subplot(1, 3, 1)
26 plt.imshow(H, cmap='gray')
27 plt.title('Canal H (Teinte)')
28 plt.axis('off')
29
30 plt.subplot(1, 3, 2)
31 plt.imshow(S, cmap='gray')
32 plt.title('Canal S (Saturation)')
33 plt.axis('off')
34
35 plt.subplot(1, 3, 3)
36 plt.imshow(V, cmap='gray')
37 plt.title('Canal V (Valeur)')
38 plt.axis('off')
39
40 plt.tight_layout()
41 plt.show()
42
```

Nous avons d'abord ouvert l'image comme précédemment, OpenCV lit les images en BGR donc nous avons effectué la conversion en espace de couleur HSV, ensuite, avons extrait les canaux H, S, V . H pour la teinte, S pour la saturation, V pour la valeur (luminosité). Finalement nous avons affiché les trois composantes, sous trois figures comme suit :



Canal H : Les couleurs similaires dans l'image d'origine apparaîtront avec des intensités de gris proches dans ce canal.

Canal S : Les zones proches du blanc dans ce canal représentent des couleurs intenses, tandis que les zones sombres représentent des couleurs ternes ou délavées.

Canal V : Cette image montre la luminosité ou l'intensité lumineuse de chaque pixel. Les pixels clairs représentent des zones lumineuses de l'image, et les pixels sombres montrent les parties sombres.

4- Dédution des dimensions de l'image dans chacun des espaces :

Les 3 espaces ont chacun trois canaux, donc l'image est globalement de dimension (Hauteur, Largeur, 3), elle est donc de dimension (362, 500, 3) dans les 3 cas.

B- Manipulation de donnée

Exercice 1

Cette image est représentée en deux couleurs uniquement : le noir (0) et le blanc (255). Elle peut donc être considérée comme une image en niveaux de gris, mais avec seulement deux niveaux distincts.



1. Construction de l'image binaire :

L'utilité des fonctions `np.ones` et `np.zeros` est de permettre la création de tableaux remplis respectivement de 1 (blanc) et de 0 (noir). Ensuite, en utilisant les fonctions `np.vstack` et `np.hstack`, on peut empiler horizontalement une ligne avec une alternance de cases blanches et noires. Après cela, cette ligne peut être empilée verticalement pour créer l'image finale du damier.

2. Calculez la taille mémoire de cette image binaire :

Pour calculer la taille mémoire d'une image binaire, il est nécessaire de connaître le type de données utilisé pour stocker chaque pixel ainsi que le nombre total de pixels.

- **Calcul du nombre total de pixels :**

L'image est composée de 8x8 carreaux, et chaque carreau fait 16x16 pixels. Le nombre total de pixels est donc :

Nombre total de pixels = $16 \times 8 \times 16 \times 8 = 128 \times 128 = 16384$ pixels

- **Type de données :**

En pratique, chaque pixel est généralement stocké dans un octet (8 bits). Ainsi, la taille mémoire est de :

Taille mémoire = $16384 \times 1 \text{ octet} = 16384 \text{ octets} = 16 \text{ Ko}$ (en divisant par 1024)

Ce résultat théorique peut être vérifié avec le code.

```
# Calcul de la taille mémoire (en octets)
taille_memoire = damier.nbytes # Renvoie la taille en octets
print(f"Taille mémoire de l'image binaire : {taille_memoire / 1024} Ko") # Conversion
```

Et on obtient la valeur théorique:

```
taille_memoire de l'image binaire : 16.0 Ko
```

3. Affichage de l'image avec `cv2.imshow`:

En utilisant les fonctions construites à la première question, on affiche l'image et on obtient le damier de taille 16*16

```

import cv2
import numpy as np
from skimage import io
from matplotlib import pyplot as plt

# Dimensions de chaque carreau (16x16 pixels)
carreau_size = 16

# Création d'un carreau blanc (rempli de 255) et d'un carreau noir (rempli de 0) avec
carreau_blanc = np.ones((carreau_size, carreau_size), dtype=np.uint8) * 255 # Blanc
carreau_noir = np.zeros((carreau_size, carreau_size), dtype=np.uint8) # Noir = 0

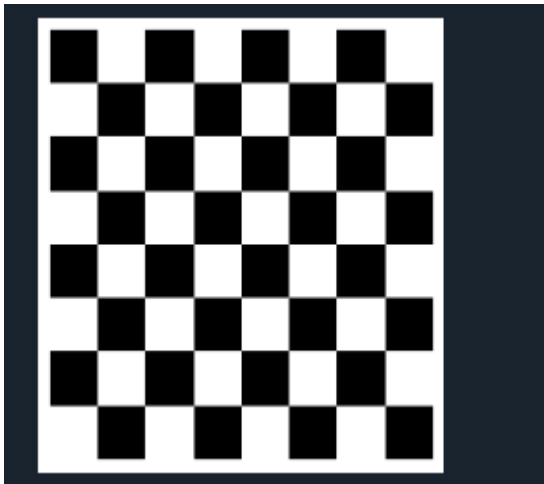
# Construction d'une ligne alternant carreaux noirs et blancs
ligne1 = np.hstack([carreau_noir, carreau_blanc] * 4)
ligne2 = np.hstack([carreau_blanc, carreau_noir] * 4)

# Empiler verticalement pour alterner les lignes
damier = np.vstack([ligne1, ligne2] * 4)

# Affichage de l'image binaire avec OpenCV
cv2.imshow("Damier", damier) # Affiche l'image dans une fenêtre
cv2.waitKey(0) # Attendre qu'une touche soit pressée
cv2.destroyAllWindows() # Ferme toutes les fenêtres ouvertes

# Calcul de la taille mémoire (en octets)
taille_memoire = damier.nbytes # Renvoie la taille en octets
print(f"Taille mémoire de l'image binaire : {taille_memoire / 1024} Ko") # Conversion

```



4. Remplacez tous les carreaux noirs de la figure 1 en carreaux blancs, puis affichez la nouvelle image.

Nous allons simplement remplacer tous les carreaux noirs (valeur 0) par des carreaux blancs (valeur 255) en utilisant le code ci-dessous. Le résultat sera une image composée uniquement de carreaux blancs de 16x16 pixels.

```

import numpy as np
import cv2
from matplotlib import pyplot as plt

# Dimensions de chaque carreau (16x16 pixels)
carreau_size = 16

# Création d'un carreau blanc (rempli de 255) et d'un carreau noir (rempli de 0) avec
carreau_blanc = np.ones((carreau_size, carreau_size), dtype=np.uint8) * 255 # Blanc
carreau_noir = np.zeros((carreau_size, carreau_size), dtype=np.uint8) # Noir = 0

# Construction d'une ligne alternant carreaux noirs et blancs
ligne1 = np.hstack([carreau_noir, carreau_blanc] * 4)
ligne2 = np.hstack([carreau_blanc, carreau_noir] * 4)

# Empiler verticalement pour alterner les lignes
damier = np.vstack([ligne1, ligne2] * 4)

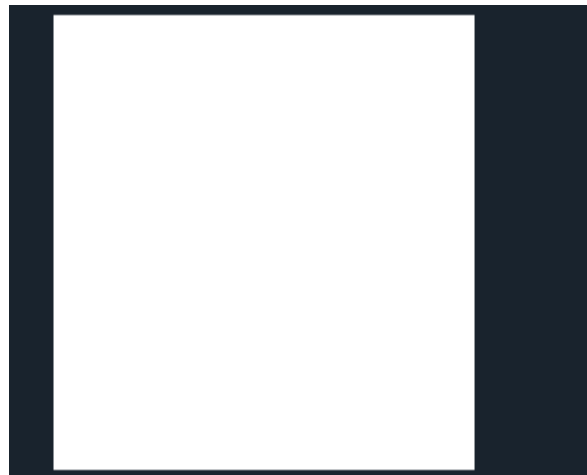
# Affichage de l'image binaire avec OpenCV
cv2.imshow("Damier", damier) # Affiche l'image dans une fenêtre
cv2.waitKey(0) # Attendre qu'une touche soit pressée
cv2.destroyAllWindows() # Ferme toutes les fenêtres ouvertes

# Calcul de la taille mémoire (en octets)
taille_memoire = damier.nbytes # Renvoie la taille en octets
print(f"Taille mémoire de l'image binaire : {taille_memoire / 1024} Ko") # Conversion

# Question 5 : Remplacement des carreaux noirs par des carreaux blancs
damier_remplace = np.where(damier == 0, 255, damier) # Remplace tous les 0 (noirs)

# Affichage de l'image modifiée avec OpenCV
cv2.imshow("Damier Modifié", damier_remplace) # Affiche l'image modifiée
cv2.waitKey(0) # Attendre qu'une touche soit pressée
cv2.destroyAllWindows() # Ferme toutes les fenêtres ouvertes

```



Exercice 2 : Image en niveau de gris

1. Représentez l'image de la figure 1 en une image en niveau de gris.

Comme pour l'image de damier, chaque pixel a une valeur en niveaux de gris, avec 0 pour le noir et 255 pour le blanc. Cela signifie que nous avons des pixels noirs et des pixels blancs. L'image est donc affichée en niveaux de gris. Nous verrons la même image sans aucune conversion.

2. Calcul de la taille mémoire de cette image

La taille mémoire de l'image binaire ou en niveaux de gris est donc toujours de 16Ko.

Exercice 3 : Image en couleur

1. Construisez une image composée de 6 imagerie de taille identique (25*25), de couleurs différentes (voir figure 2) np.ones, np.zeros, np.concatenate, np.vstack, np.hstack. Des informations sur les composantes RGB sont données dans le tableau 1.

Alors pour faire ceci Composantes RGB fournies (Tableau 1) :

Blanc : (R=1, G=1, B=1)

Rouge : (R=1, G=0, B=0)

Jaune : (R=1, G=1, B=0)

Vert : (R=0, G=1, B=0)

Noir : (R=0, G=0, B=0)

Bleu : (R=0, G=0, B=1)

Nous allons d'abord créer des imagerie en couleur en utilisant les matrices NumPy et les valeurs RGB correspondantes. Ensuite, en utilisant np.hstack et np.vstack, nous rassemblerons ces imagerie en une seule image.

```

import numpy as np
import matplotlib.pyplot as plt

# Taille de chaque imagette (25x25 pixels)
imagette_size = 25

# Création des imagettes avec leurs couleurs en RGB (chaque canal R, G, B)
blanc = np.ones((imagette_size, imagette_size, 3), dtype=np.uint8) * 255 # Blanc
rouge = np.zeros((imagette_size, imagette_size, 3), dtype=np.uint8)
rouge[:, :, 0] = 255 # Rouge (1,0,0)
jaune = np.zeros((imagette_size, imagette_size, 3), dtype=np.uint8)
jaune[:, :, 0:2] = 255 # Jaune (1,1,0)
vert = np.zeros((imagette_size, imagette_size, 3), dtype=np.uint8)
vert[:, :, 1] = 255 # Vert (0,1,0)
noir = np.zeros((imagette_size, imagette_size, 3), dtype=np.uint8) # Noir (0,0,0)
bleu = np.zeros((imagette_size, imagette_size, 3), dtype=np.uint8)
bleu[:, :, 2] = 255 # Bleu (0,0,1)

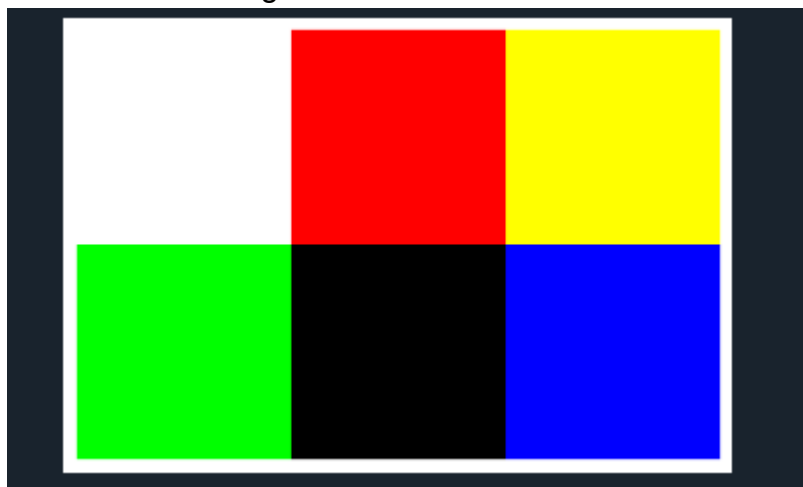
# Combiner les imagettes en utilisant np.hstack et np.vstack
ligne1 = np.hstack([blanc, rouge, jaune])
ligne2 = np.hstack([vert, noir, bleu])

# Assembler les lignes pour former l'image finale
image_finale = np.vstack([ligne1, ligne2])

# Affichage de l'image
plt.imshow(image_finale)
plt.axis('off') # Cacher les axes
plt.show()
# Calcul de la taille mémoire
taille_memoire = image_finale.nbytes # Taille en octets
print(f"Taille mémoire de l'image en couleur : {taille_memoire / 1024} Ko")

```

Et on obtient l'image suivante:



2. Calculez la taille mémoire de cette image en couleur.

On sait que chaque imagettes a une taille de 25*25 pixels et on a en tout 3 imagette par ligne et 2 ligne

Et donc la dimension totale de l'image : $25*2*25*3=50*70$ pixels=3750 pixels

Et comme on a 3 canaux, chaque pixel occupe 3 octets(on a 1 octets par canal)

Alors la taille de mémoire de cette image est de $3750\text{pixels}*3\text{octes}=11.250$ octets
=11.25Ko

En exécutant le code sur python on obtient un résultat de 10.986328125 Ko

```
In [12]: runfile('C:/Users/greg7/.spyder-py3/exo3.py', w
Taille mémoire de l'image en couleur : 10.986328125 Ko
```

Et c'est parce que simplement 1 Ko est égal à 1024 octets, et non 1000 octets

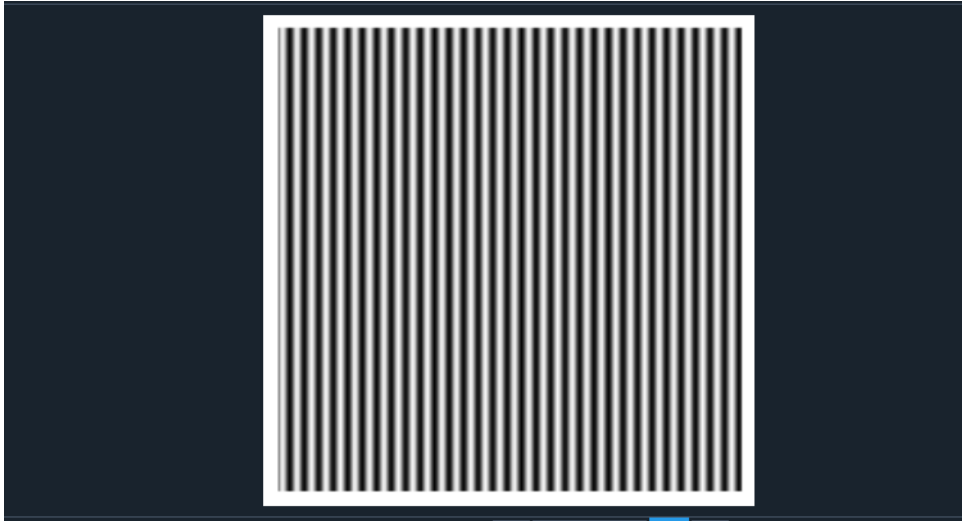
Exercice 4 : Image sinus 2D – Transformée de Fourier

Génération de l'image :

Nous avons utilisé le code ci-dessous :

```
1  # -*- coding: utf-8 -*-
2  """
3  Created on Mon Sep 30 14:17:34 2024
4
5  @author: 12108993
6  """
7
8  import cv2
9  from matplotlib import pyplot as plt
10 import numpy as np
11
12
13
14 # Génération de valeurs de 0 à 255
15 x = np.arange(256)
16
17 # Division pour ajuster la fréquence du sinus
18 d = 500 # Vous pouvez ajuster cette valeur pour changer la fréquence du sinus
19
20 # Calcul du sinus de x
21 y = np.sin(2 * np.pi * x / d)
22
23 # Ajout d'un offset pour supprimer les valeurs négatives
24 y += np.abs(min(y)) # On ajoute l'offset pour que toutes les valeurs soient positives
25
26 # Création de l'image 256x256 en répétant les valeurs de y
27 img = np.array([y[j] * 127 for j in range(256)] for i in range(256)], dtype=np.uint8)
28
29 # Affichage de l'image
30 plt.imshow(img, cmap='gray')
31 plt.axis('off') # Retirer les axes pour une meilleure visualisation
32 plt.show()
33
```

Avec ce code nous avons généré un tableau 1d de valeurs 0 à 255 pour les axes X, nous avons ensuite calculer la fonctions sinus pour chaque valeur de X. Nous avons créé une variable d qui nous a permis d'ajuster la fréquence, la valeur de d détermine la fréquence des oscillations du sinus dans l'image. Plus d est petit, plus la fréquence est élevée, ce qui crée des bandes fines et rapprochées dans l'image. Inversement lorsque d est grand, la fréquence est plus basse, produisant donc des bandes larges et espacées. Nous avons ensuite ajouté un offset pour s'assurer que les valeurs sont positives. Enfin nous avons générer des images :



1- Analyse de la transformée de Fourier de cette image :

Le code proposé réalise une analyse de la transformée de Fourier d'une image à l'aide de la bibliothèque OpenCV et Matplotlib. La fonction `cv2.dft()` est utilisée pour calculer la transformée de Fourier discrète (TFD) de l'image.

Le paramètre `cv2.DFT_COMPLEX_OUTPUT` assure que la sortie est en nombre complexe (composée de parties, réelle et imaginaire).

La fonction `np.fft.fftshift()` déplace les basses fréquences de la transformée de Fourier au centre de l'image.

`cv2.magnitude()` calcule à partir des composantes réelles et imaginaires de la Transformée de Fourier, le spectre de magnitude (le spectre de fréquences d'une image après l'application de la transformée de Fourier)

2- Analyse de l'impact du paramètre d :

« d » nous a permis d'ajuster la fréquence, la valeur de d détermine la fréquence des oscillations du sinus dans l'image. Plus d est petit, plus la fréquence est élevée, ce qui crée des bandes fines et rapprochées dans l'image. Inversement lorsque d est grand, la fréquence est plus basse, produisant donc des bandes larges et espacées :

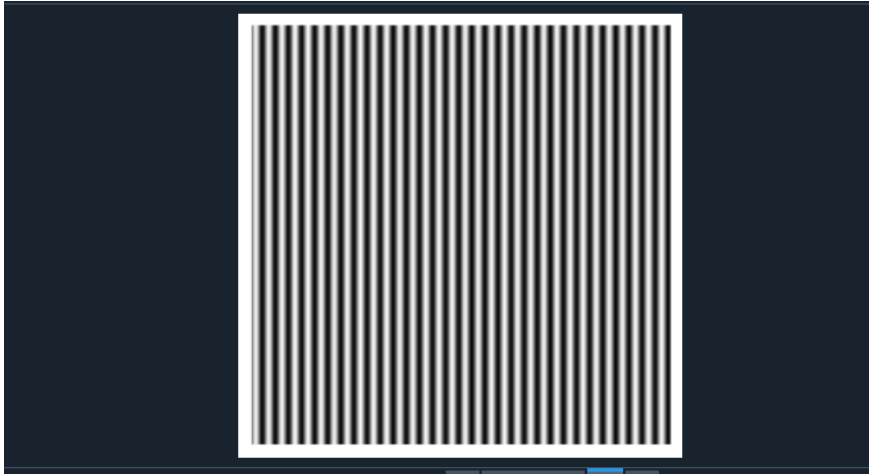


Figure 1: $d = 8$



Figure 2 : $d = 500$

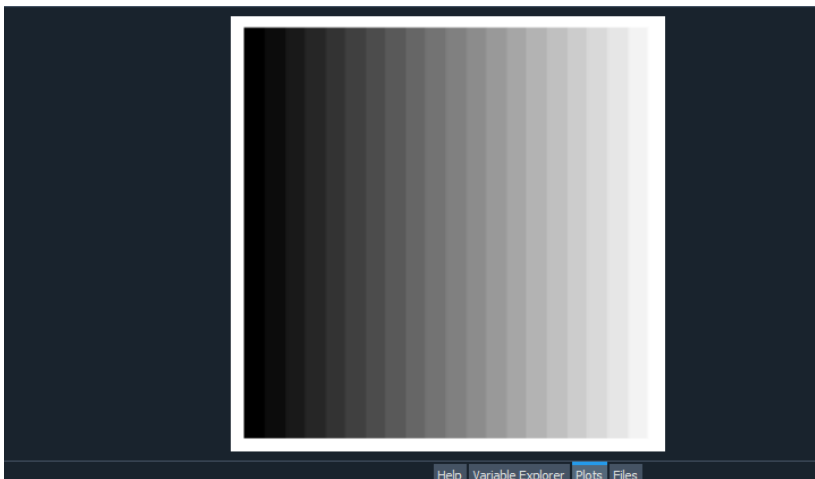
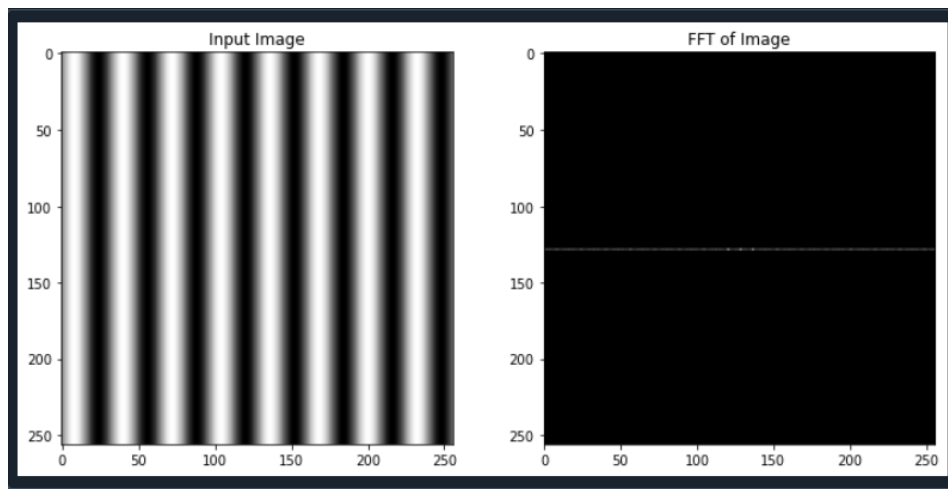


Figure 3 : $d = 1000$

3- Calcul de la Transformée de Fourier



Commentaire du résultat :

Lorsque nous effectuons la transformée de Fourier d'une image générée par un sinus 2D avec une fréquence déterminée ($d=32$), le spectre de Fourier révèle des pics qui correspondent aux fréquences présentes dans l'image. Dans ce cas, les trois pics que nous avons observé dans le spectre de Fourier sont liés aux caractéristiques périodiques du motif sinusoïdal :

Pic central (fréquence nulle) : Ce pic représente la composante continue de l'image, indiquant la moyenne des intensités lumineuses. Dans ce cas, il est le reflet de l'offset que nous avons ajouté pour rendre toutes les valeurs positives.

Pics latéraux : Les deux autres pics représentent les fréquences harmoniques du motif sinusoïdal. Ces pics indiquent les variations de fréquence qui correspondent à l'oscillation du sinus. La distance entre le pic central et ces pics latéraux correspond à la fréquence fondamentale de l'oscillation, tandis que la position des pics communique des informations sur la direction et la périodicité de la variation des intensités dans l'image.

Ainsi , les trois pics dans le spectre de Fourier montrent comment le motif sinusoïdal est constitué, avec un pic pour la composante constante et deux autres pour les variations périodiques.

C- Transformation des images

Exercice 1 :

1- Les dimensions de l'image :

```
# Importer les bibliothèques nécessaires
from skimage import io
from skimage.transform import downscale_local_mean
import matplotlib.pyplot as plt

# Charger l'image
image_path = 'lenna512.bmp' # Remplace par le chemin de ton image
img = io.imread('lenna512.bmp')

# Obtenir les dimensions de l'image
print(f"Dimensions de l'image : {img.shape}")
```

Ce code : Importe des bibliothèques pour lire, transformer et afficher des images. Ensuite charge une image (ici lenna512.bmp) à partir de son chemin avec `io.imread()`. Finalement il affiche les dimensions de l'image en imprimant sa taille avec `img.shape`.

Nous obtenons ce résultat :

```
In [54]: runfile('C:/Users/12108993/...')
Dimensions de l'image : (512, 512)
```

2- Échantillonnage de l'image par un facteur 2 selon chacune des directions :

```
# Obtenir les dimensions de l'image
print(f"Dimensions de l'image : {img.shape}")

# Échantillonner l'image par un facteur de 2 dans chaque direction
img_downscaled = downscale_local_mean(img, (2, 2))
# calculer les dimensions de l'image sous-échantillonnée
downscaled_shape = (original_shape[0] // 2, original_shape[1] // 2)
print(f"Dimensions de l'image sous-échantillonnée : {downscaled_shape}")
# Afficher l'image réduite
plt.imshow(img_downscaled, cmap='gray')
plt.axis('off') # Masquer les axes
# plt.title("Image réduite par un facteur de 2")
plt.show()
```

L'échantillonnage d'une image consiste à réduire sa taille en sélectionnant un sous-ensemble des pixels d'origine. Dans le code, nous faisons cela en utilisant la fonction `downscale_local_mean`, qui effectue un sous-échantillonnage tout en conservant une moyenne locale des pixels dans chaque bloc. Cela permet de préserver une certaine qualité visuelle, car au lieu de simplement supprimer des pixels, on moyenne les blocs de pixels voisins.

Pour échantillonner l'image selon 2 directions il faut qu'elle soit en 2d, donc il faut la charger avec la fonction `io.imread`

Le calcul de la taille de l'image sous-échantillonnée :

Théoriquement : Dimension de l'image sous-échantillonnée la n ème fois = Dimension de l'image initiale / 2^n

Donc pour une image sous-échantillonnée Dim = $(512/2, 512/2) = (256, 256)$

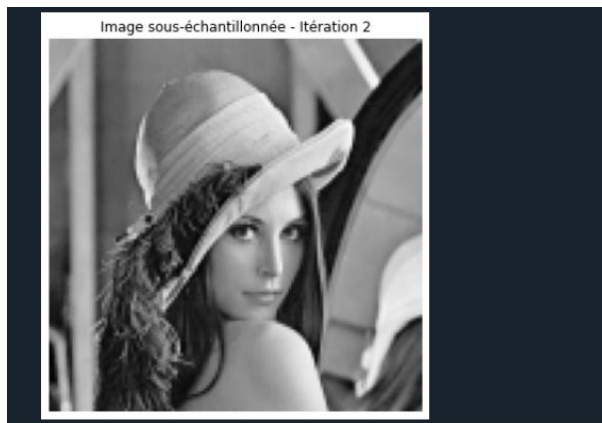
Nous avons confirmé ceci avec le code :

```
In [57]: runfile('C:/Users/12108993/Desktop/Suite TP/unt...  
Dimensions de l'image : (512, 512)  
Dimensions de l'image sous-echantillonee : (256, 256)
```

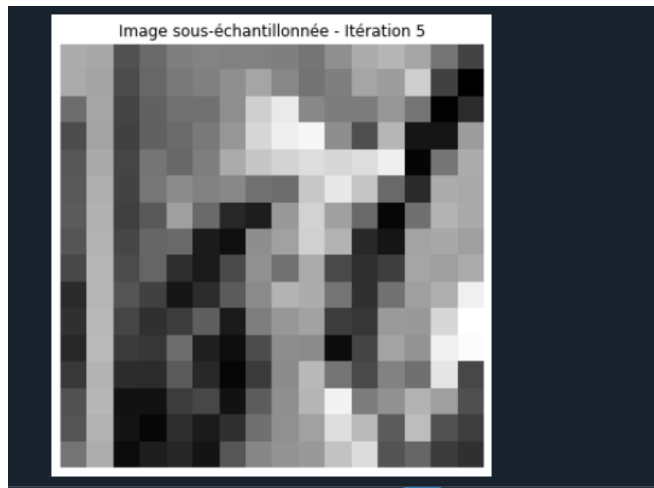
3- Réitération du processus d'échantillonnage :

Les images obtenues après itération :

2 itérations :



5 itérations :



Nous observons une dégradation de la qualité de l'image ce qui se voit dans la dimension de l'image :

```
Dimensions de l'image originale : (512, 512)
Dimensions après itération 1 : (256, 256)
Dimensions après itération 2 : (128, 128)
Dimensions après itération 3 : (64, 64)
Dimensions après itération 4 : (32, 32)
Dimensions après itération 5 : (16, 16)
```

On en déduit donc que lorsque le facteur d'échantillonnage est grand, plus de pixels d'origine sont condensés en un seul, ce qui entraîne une perte significative de détails. Cela se traduit par une image floue ou pixellisée.

4- Autre échantillonnage :

```
# Importer les bibliothèques nécessaires
from skimage import io
from skimage.transform import downscale_local_mean
import matplotlib.pyplot as plt
from skimage.transform import rescale

# Charger l'image avec io.imread
image_path = 'lenna512.bmp' # Remplace par le chemin de ton image
img = io.imread(image_path)

# Sous-échantillonner l'image avec un facteur de 1/4
img_rescaled = rescale(img, 1.0 / 4.0, anti_aliasing=False)

# Afficher l'image sous-échantillonnée
plt.imshow(img_rescaled, cmap='gray')
plt.axis('off') # Masquer les axes
plt.title("Image sous-échantillonnée par rescale avec facteur 1/4")
plt.show()
```

La fonction `rescale` : réduit la taille de l'image par un facteur de 4 (c'est-à-dire qu'elle divise la taille des dimensions par 4).

anti_aliasing=False : Signifie qu'il n'y a pas de lissage des bords pendant l'échantillonnage. Cela peut entraîner des effets de "pixellisation", car les hautes fréquences ne sont pas adoucies.



La différence avec la méthode précédente :

- Avec `downscale_local_mean`, l'échantillonnage réduit la taille de l'image tout en conservant une certaine qualité en calculant une moyenne locale des pixels, ce qui rend l'image plus fluide.
- Avec `rescale` sans anti-aliasing, l'image est simplement redimensionnée sans aucun lissage, ce qui peut entraîner des effets d'aliasing et une qualité visuelle plus dégradée, notamment avec des bords moins lisses et donc plus pixelisés.

5- Commentaire sur une autre possibilité de redimensionner :

```
# Importer les bibliothèques nécessaires
from skimage import io
from skimage.transform import resize
import matplotlib.pyplot as plt

# Charger l'image avec io.imread
image_path = 'lenna512.bmp' # Remplace par le chemin de ton image
img = io.imread(image_path)

# Redimensionner l'image à 160x160 avec anti-crênelage
img_resize = resize(img, (160, 160), anti_aliasing=True)

# Afficher l'image redimensionnée
plt.imshow(img_resize, cmap='gray')
plt.axis('off') # Masquer les axes
plt.title("Image redimensionnée à 160x160 avec anti-aliasing")
plt.show()

# Afficher les dimensions de l'image redimensionnée
print(f"Dimensions de l'image redimensionnée : {img_resize.shape}")
```

La différence dans ce cas est que l'anti-aliasing est activé avec `anti_aliasing = True`, ceci permet de lisser les bords lors du redimensionnement. Donc la qualité visuelle est améliorée.



Exercice 2 : Quantification

1-

L'image « lenna512.bmp » est codée sur 215 niveaux de gris. Ce chiffre se situe entre 2727 (128) et 2828 (256). Cela signifie que l'image utilise une palette de niveaux de gris qui est légèrement inférieure à la capacité maximale d'une image en 8 bits, qui est de 256 niveaux.

```
import cv2

# Lire l'image en niveaux de gris
image = cv2.imread('lenna512.bmp', cv2.IMREAD_GRAYSCALE)

# Obtenir le type de données et le nombre de niveaux
unique_values = len(set(image.flatten()))
print(f"Nombre de niveaux de gris : {unique_values}")
```

```
In [73]: runfile('C:/Users/12108993/Desktop/S
Nombre de niveaux de gris : 215
```

2- L'effet de quantification sur l'image :

Nous avons genre ce code pour avoir la quantification de lenna.512 sur différents niveaux

Created on Mon Sep 30 16:23:17 2024

@author: 12108993

"""

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Lire l'image en niveaux de gris
image = cv2.imread('lenna512.bmp', cv2.IMREAD_GRAYSCALE)

# Fonction de quantification de l'image
def quantize_image(image, levels):
    # Calculer le facteur de quantification
    quantization_factor = 256 // levels
    # Quantifier l'image
    quantized_image = (image // quantization_factor) * quantization_factor
    return quantized_image

# Niveaux de quantification
levels_list = [128, 64, 32, 16, 8, 2]

# Préparer la figure pour l'affichage
plt.figure(figsize=(12, 8))

# Afficher l'image originale
plt.subplot(2, 4, 1)
plt.imshow(image, cmap='gray', vmin=0, vmax=255)
plt.title('Image Originale')
plt.axis('off')

# Afficher les images quantifiées
for i, levels in enumerate(levels_list):
    quantized_image = quantize_image(image, levels)
    plt.subplot(2, 4, i + 2)
    plt.imshow(quantized_image, cmap='gray', vmin=0, vmax=255)
    plt.title(f'Quantification: {levels} niveaux')
    plt.axis('off')

# Afficher la figure
plt.tight_layout()
plt.show()
```



3. Calculez l'erreur quadratique moyenne de quantification (entre l'image originale et l'image avec la nouvelle quantification):

Nous allons calculer l'erreur quadratique moyenne pour chacun des niveaux

```
In [81]: runfile('C:/Users/121
EQM pour 128 niveaux : 0.50
EQM pour 64 niveaux : 3.50
EQM pour 32 niveaux : 17.46
EQM pour 16 niveaux : 78.04
EQM pour 8 niveaux : 94.43
EQM pour 2 niveaux : 105.83
```

```
# Afficher les images quantifiées et calculer l'EQM
for i, levels in enumerate(levels_list):
    quantized_image = quantize_image(image, levels)
    mse = calculate_mse(image, quantized_image)
    mse_values.append(mse)

plt.subplot(2, 4, i + 2)
plt.imshow(quantized_image, cmap='gray')
plt.title(f'Quantification: {levels} niveaux\nEQM: {mse:.2f}')
plt.axis('off')
```



4. Tracer l'erreur quadratique moyenne en fonction des différents pas de quantification.

Nous nous attendons à avoir une petite erreur quadratique moyenne (EQM) pour un petit pas de quantification, et une grande EQM pour un pas de quantification élevé.

Ainsi, plus le nombre de niveaux de quantification est élevé, plus l'erreur quadratique moyenne est faible.

```

@author: 12108993
"""

import cv2
import numpy as np
import matplotlib.pyplot as plt

# Lire l'image en niveaux de gris
image = cv2.imread('lenna512.bmp', cv2.IMREAD_GRAYSCALE)

# Fonction de quantification de l'image
def quantize_image(image, levels):
    quantization_factor = 256 // levels
    quantized_image = (image // quantization_factor) * quantization_factor
    return quantized_image

# Fonction pour calculer l'erreur quadratique moyenne
def calculate_mse(original, quantized):
    mse = np.mean((original - quantized) ** 2)
    return mse

# Niveaux de quantification
levels_list = [128, 64, 32, 16, 8, 2]
mse_values = []
quantization_steps = []

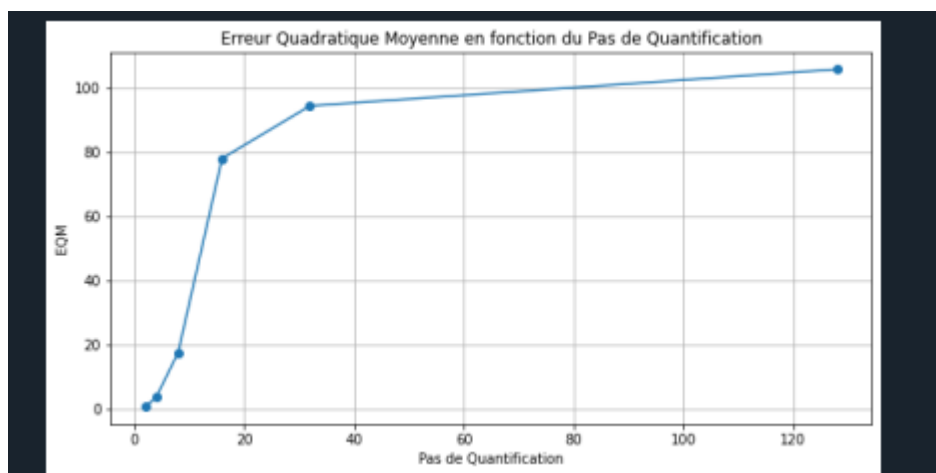
# Calculer l'EQM et le pas de quantification pour chaque niveau
for levels in levels_list:
    quantized_image = quantize_image(image, levels)
    mse = calculate_mse(image, quantized_image)
    mse_values.append(mse)

    # Calculer le pas de quantification
    step_size = 256 / levels
    quantization_steps.append(step_size)

# Tracer l'EQM en fonction du pas de quantification
plt.figure(figsize=(10, 5))
plt.plot(quantization_steps, mse_values, marker='o')
plt.title('Erreur Quadratique Moyenne en fonction du Pas de Quantification')
plt.xlabel('Pas de Quantification')
plt.ylabel('EQM')
plt.xscale('log') # Optionnel : Utiliser une échelle logarithmique pour le pas
plt.grid(True)
plt.show()

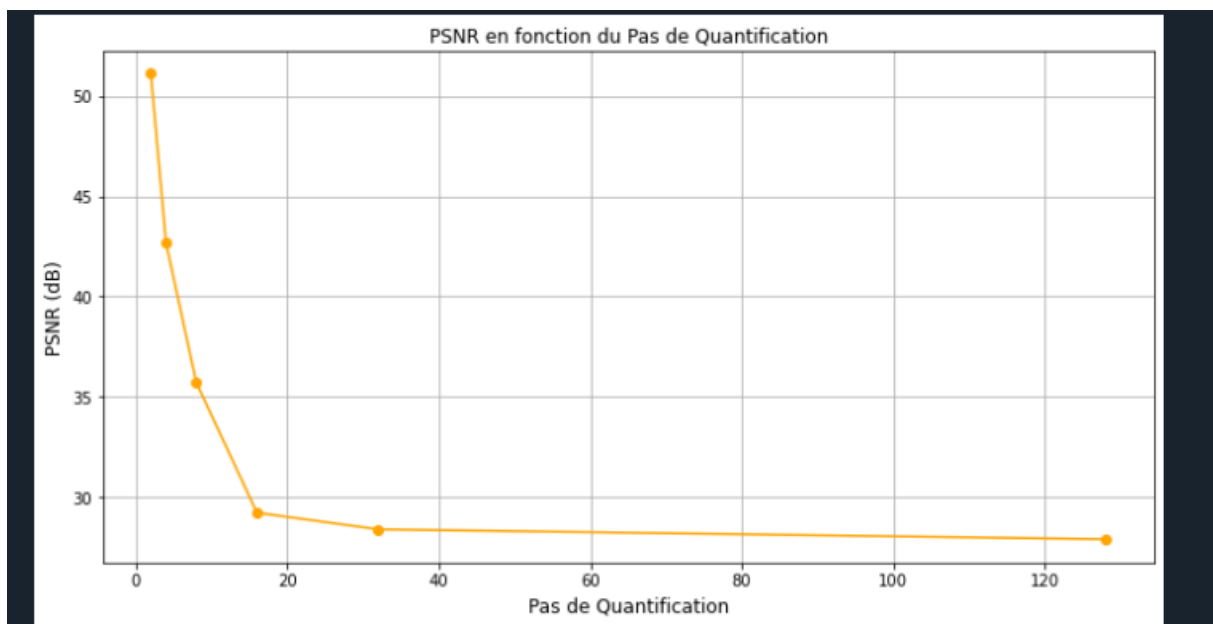
# Afficher les valeurs d'EQM
for step, mse in zip(quantization_steps, mse_values):
    print(f"EQM pour un pas de quantification de {step:.2f} : {mse:.2f}")

```



L'erreur quadratique moyenne (EQM) augmente à mesure que le pas de quantification devient plus grand, ce qui est logique. En effet, avec un grand pas de quantification, l'image est codée avec moins de niveaux de gris, ce qui réduit la précision et dégrade la qualité de l'image.

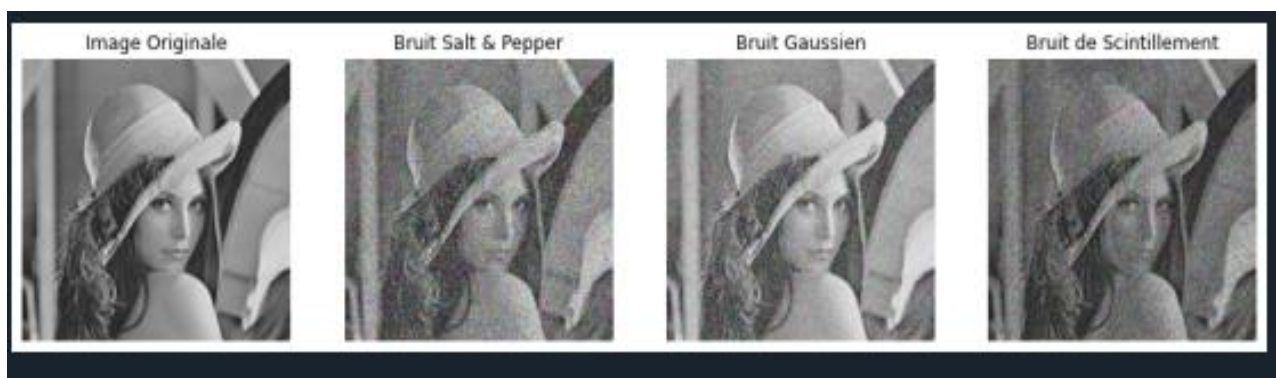
Ainsi, nous nous intéressons au PSNR pour chaque niveau de quantification, et nous avons tracé l'évolution du PSNR en fonction des pas de quantification



Et comme nous l'avons déduit auparavant, plus le pas de quantification est petit, plus le PSNR est grand, et donc meilleure est la qualité de l'image.

D. Impact du bruit sur les images

Nous allons essayer de voir l'impact des différents types de bruit sur l'image Lenna et voici le résultat qu'on avait obtenue en utilisant le code donne



Commentaire sur chacune des images

- **Image Originale** : L'image est nette et de bonne qualité
- **Bruit Salt & Pepper** : on observe l'apparition des points noirs et blancs aléatoires perturbent l'image.
- **Bruit Gaussien** : Un grain plus uniforme est visible, réparti de manière aléatoire, floutant légèrement l'image.
- **Bruit de Scintillement** : L'image est affectée par des taches aléatoires dues à la multiplication du bruit avec l'image originale, donnant un effet plus granuleux.

En générale, on observe une dégradation de la qualité par rapport à l'image originale

Le PSNR est donne par la formule :

$$\text{PSNR} = 10 \log_{10} \left(\frac{(2^b - 1)^2}{\text{EQM}} \right)$$

Et voici le PSNR pour chacune des images

```
PSNR Salt & Pepper: 10.674751823191897
PSNR Gaussien: 13.246565913920135
PSNR Scintillement: 12.17464360450867
```

Commentaire et analyse:

Bruit Salt & Pepper: Ce type de bruit présente le PSNR le plus bas, ce qui signifie une qualité d'image fortement dégradée. Cela s'explique par le fait que ce bruit introduit des pixels noirs et blancs aléatoires, créant un fort contraste. Ce contraste réduit considérablement la qualité visuelle de l'image.

Bruit gaussien : L'image est légèrement moins dégradée par rapport au bruit Salt & Pepper. Ce type de bruit est réparti de manière plus uniforme, créant un flou subtil qui n'affecte pas autant les zones distinctes de l'image.

Bruit de scintillement : Le PSNR pour ce bruit est intermédiaire entre les deux autres. Ce type de bruit multiplie des variations aléatoires avec l'image, créant un effet granuleux. Cela dégrade l'image, mais de façon moins brutale que le bruit Salt & Pepper.

Conclusion :

Le traitement d'images a permis d'explorer différentes techniques essentielles, telles que la lecture et la manipulation d'images en Python, l'analyse des espaces de couleurs, et la quantification. Nous avons utilisé des bibliothèques comme NumPy, SciPy et OpenCV pour réaliser des opérations telles que le redimensionnement et l'ajout de bruit, tout en étudiant leur impact sur la qualité des images. L'analyse des images bruitées a révélé l'importance du rapport signal à bruit (PSNR) pour évaluer la clarté des données traitées.